DATA ENGINEERING PORTFOLIO

Customer Shopping Behavior Analytics and BI Pipeline

End-to-end workflow using Python, PostgreSQL, DBeaver, and Power BI.

Python | Pandas | SQLAlchemy | PostgreSQL | DBeaver | Power BI

3,900	$59.76	3.75/5	0
Customers	Avg Purchase	Avg Rating	Missing Values After Cleaning

Prepared for recruiters and engineering managers

Executive Summary

This project delivers an end-to-end customer shopping behavior analytics solution. The goal was to analyze revenue, demographics, subscription status, customer ratings, and purchase frequency by building a complete analytics engineering workflow from raw CSV ingestion to an interactive Power BI dashboard.

The solution demonstrates the ability to design and implement a practical data pipeline using Python, Pandas, SQLAlchemy, PostgreSQL, DBeaver, SQL, and Power BI. The project goes beyond notebook-level analysis by storing cleaned data in a relational database and using it as the source for BI reporting.

3,900	$59.76	3.75/5	0
Total Customers	Avg Purchase Amount	Avg Review Rating	Missing Values After Cleaning

Project objective: Analyze customer shopping behavior to identify business trends in revenue, demographics, category performance, subscription status, and customer engagement patterns.

Core outcomes delivered:

- Cleaned and standardized raw customer shopping data using Python and Pandas.
- Applied category-wise median imputation to handle missing review ratings with business context.
- Created engineered features such as age_group and purchase_frequency_days.
- Loaded the cleaned DataFrame into PostgreSQL using SQLAlchemy and secure URL.create connection design.
- Performed advanced SQL analysis in DBeaver using aggregations, CTEs, and window functions.
- Connected Power BI Desktop directly to PostgreSQL and created an interactive dashboard with KPI cards, charts, and slicers.

The final workflow represents a complete analytics lifecycle: raw data ingestion, transformation, database architecture, SQL exploration, BI visualization, and business storytelling.

Architecture & Tech Stack

CSV -> Python -> PostgreSQL -> Power BI

The project follows a layered architecture where each tool has a clear role in the data lifecycle. Python performs the transformation layer, PostgreSQL acts as the storage and serving layer, DBeaver supports analytical SQL development, and Power BI provides the business-facing reporting layer.

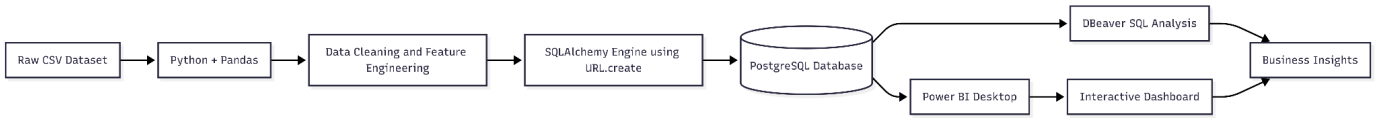


Figure 1: Data pipeline architecture screenshot showing the CSV to Python, PostgreSQL, DBeaver, Power BI, and Business Insights flow.

Layer	Tool / Technology	Purpose
Raw Source	CSV Dataset	Customer shopping behavior source data.
Data Processing	Python + Pandas	Load, clean, transform, and engineer features.
Database Connectivity	SQLAlchemy + psycopg2	Create Python-to-PostgreSQL connection and load DataFrame.
Secure Connection Pattern	SQLAlchemy URL.create	Avoid brittle manual connection strings and improve maintainability.
Database	PostgreSQL	Store cleaned analytics table for downstream use.
SQL IDE	DBeaver	Write CTEs, window functions, and aggregation queries.
Visualization	Power BI Desktop	Build interactive dashboard from PostgreSQL import.

This structure mirrors a production-style analytics workflow where raw files are not directly used for dashboards. Instead, data is cleaned, structured, stored, queried, and then consumed by a BI layer.

Data Engineering Pipeline

Python and Pandas Transformation Layer

The raw CSV was loaded into a Pandas DataFrame and transformed into a clean analytics-ready dataset. This stage focused on data quality, schema consistency, feature creation, and business-friendly fields.

1. Column Name Standardization

All columns were converted to lowercase snake_case to make Python code, SQL queries, and Power BI fields easier to read and maintain.

```
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(r"[^a-z0-9]+", "_", regex=True)
    .str.strip("_")
)
```

Examples: Customer ID -> customer_id, Purchase Amount (USD) -> purchase_amount_usd, Review Rating -> review_rating.

2. Missing Value Handling with Category Medians

The review_rating column had missing values. Instead of using one global median, missing values were filled using the median rating within each product category. This keeps the imputation closer to the behavior of that specific category.

```
df['review_rating'] = df.groupby('category')['review_rating'].transform(
    lambda x: x.fillna(x.median())
)
```

3. Feature Engineering

Feature	Method	Business Purpose
age_group	pd.qcut on age with four labels	Analyze revenue and customer behavior by demographic segment.
purchase_frequency_days	Dictionary mapping from text frequency to day values	Convert purchase cadence into a numeric feature for analysis.
cleaned review_rating	Group-wise median imputation	Preserve category-level rating behavior while removing missing values.

```
frequency_mapping = {
    'Weekly': 7,
    'Fortnightly': 14,
    'Bi-Weekly': 14,
    'Monthly': 30,
    'Quarterly': 90,
    'Every 3 Months': 90,
    'Annually': 365
}

df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
```

Database Architecture

SQLAlchemy + PostgreSQL

After transformation, the cleaned DataFrame was pushed directly into a local PostgreSQL database. PostgreSQL became the central analytical storage layer for SQL exploration and Power BI reporting.

The project used SQLAlchemy with URL.create to construct the database connection in a cleaner and more secure way than manually concatenating a raw connection string.

```
from sqlalchemy import create_engine
from sqlalchemy.engine import URL

connection_url = URL.create(
    drivername="postgresql+psycopg2",
    username="postgres",
    password="<password>",
    host="localhost",
    port=5432,
    database="shopping_db"
)

engine = create_engine(connection_url)

df.to_sql(
    name="customer_shopping_behavior",
    con=engine,
    if_exists="replace",
    index=False
)
```

Database design benefits:

- Creates a structured database layer between raw files and BI dashboards.
- Allows repeatable SQL analysis using a clean PostgreSQL table.
- Supports downstream tools such as DBeaver and Power BI through standard database connectivity.
- Makes the workflow closer to real analytics engineering and data warehouse patterns.

Metric	Value
Raw rows	3,900
Raw columns	18
Final columns after feature engineering	20
Missing values after cleaning	0

Business Insights & SQL Analysis

DBeaver SQL Layer

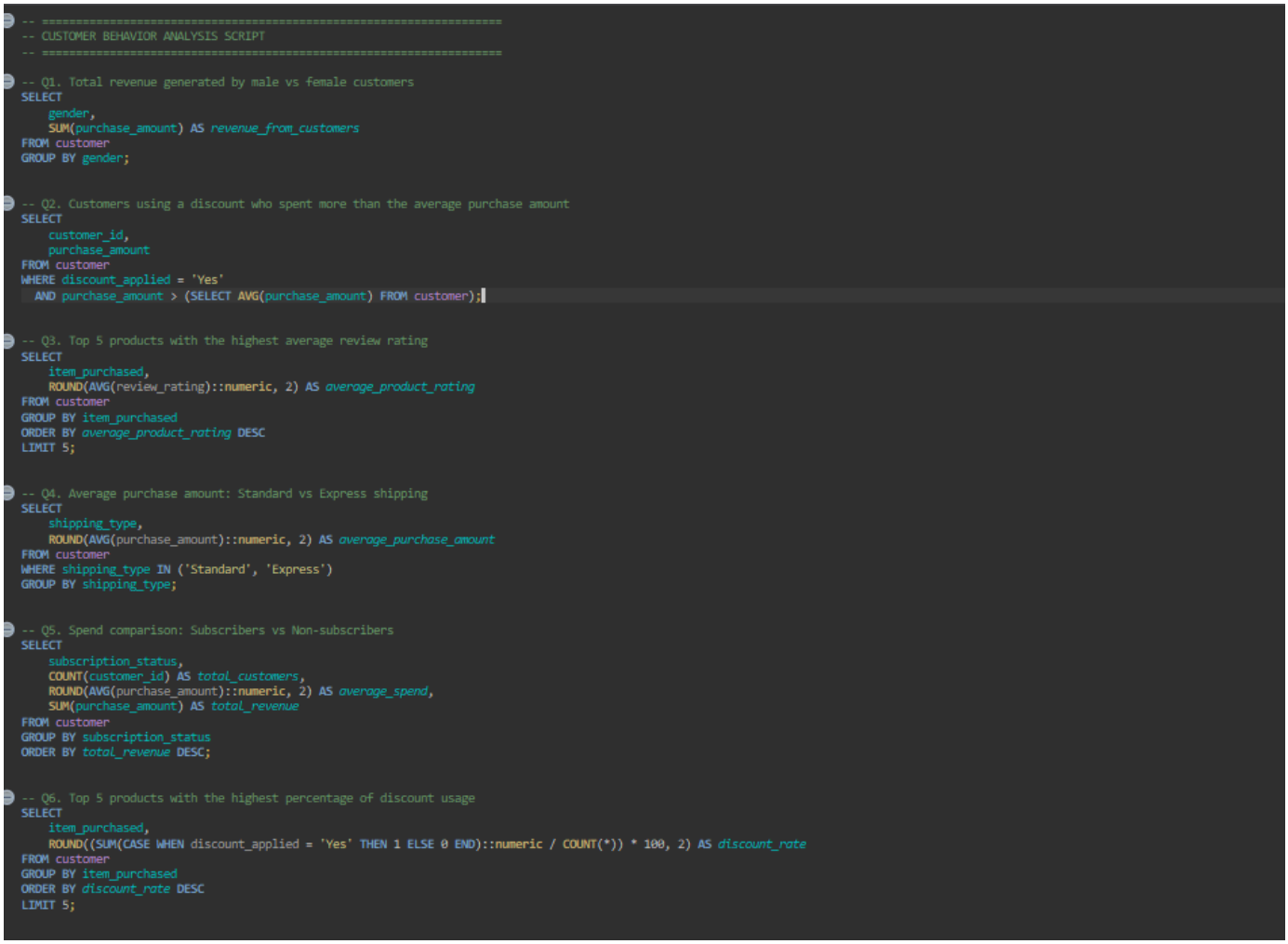
Once the cleaned data was available in PostgreSQL, advanced SQL analysis was performed in DBeaver. The SQL layer was used to validate the data and generate business insights before building the dashboard.

SQL techniques demonstrated:

- Aggregations: Calculated total revenue, average purchase amount, total customers, and average review rating.
- CTEs: Structured multi-step business logic into readable query blocks.
- Window functions: Ranked products and categories without losing analytical context.
- Segment analysis: Compared revenue by subscription status, category, age group, and product.

```
WITH product_revenue AS (
    SELECT
        category,
        item_purchased,
        SUM(purchase_amount_usd) AS total_revenue,
        COUNT(*) AS total_orders
    FROM customer_shopping_behavior
    GROUP BY category, item_purchased
), ranked_products AS (
    SELECT
        category,
        item_purchased,
        total_revenue,
        total_orders,
        RANK() OVER (
            PARTITION BY category
            ORDER BY total_revenue DESC
        ) AS revenue_rank
    FROM product_revenue
)
SELECT *
FROM ranked_products
WHERE revenue_rank <= 3
ORDER BY category, revenue_rank;
```

The query above demonstrates how CTEs and window functions can be combined to identify the top revenue-generating products inside each category.



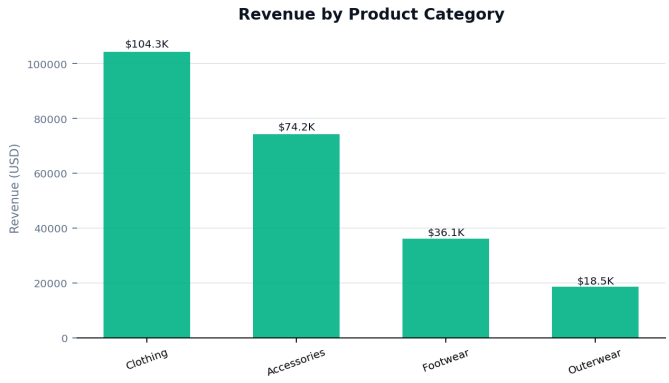
```sql
-- ============================================================
-- CUSTOMER BEHAVIOR ANALYSIS SCRIPT
-- ============================================================

-- Q1. Total revenue generated by male vs female customers
SELECT
    gender,
    SUM(purchase_amount) AS revenue_from_customers
FROM customer
GROUP BY gender;


-- Q2. Customers using a discount who spent more than the average purchase amount
SELECT
    customer_id,
    purchase_amount
FROM customer
WHERE discount_applied = 'Yes'
  AND purchase_amount > (SELECT AVG(purchase_amount) FROM customer);


-- Q3. Top 5 products with the highest average review rating
SELECT
    item_purchased,
    ROUND(AVG(review_rating)::numeric, 2) AS average_product_rating
FROM customer
GROUP BY item_purchased
ORDER BY average_product_rating DESC
LIMIT 5;


-- Q4. Average purchase amount: Standard vs Express shipping
SELECT
    shipping_type,
    ROUND(AVG(purchase_amount)::numeric, 2) AS average_purchase_amount
FROM customer
WHERE shipping_type IN ('Standard', 'Express')
GROUP BY shipping_type;


-- Q5. Spend comparison: Subscribers vs Non-subscribers
SELECT
    subscription_status,
    COUNT(customer_id) AS total_customers,
    ROUND(AVG(purchase_amount)::numeric, 2) AS average_spend,
    SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY subscription_status
ORDER BY total_revenue DESC;


-- Q6. Top 5 products with the highest percentage of discount usage
SELECT
    item_purchased,
    ROUND((SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END)::numeric / COUNT(*)) * 100, 2) AS discount_rate
FROM customer
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;
```

Figure 2: DBeaver SQL analysis screenshot showing customer behavior queries, aggregations, filters, and business exploration logic.

Business Insight Visuals

Computed from the Cleaned Dataset

The following visual summaries are generated from the cleaned customer shopping dataset. They can support the Power BI dashboard story and show the business value of the data pipeline.



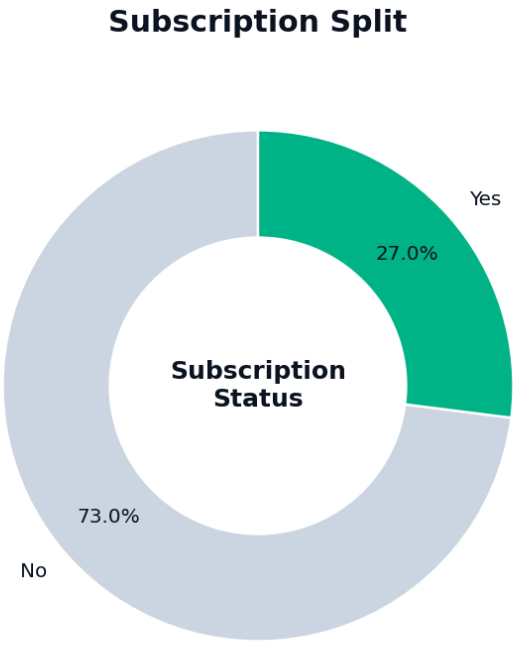
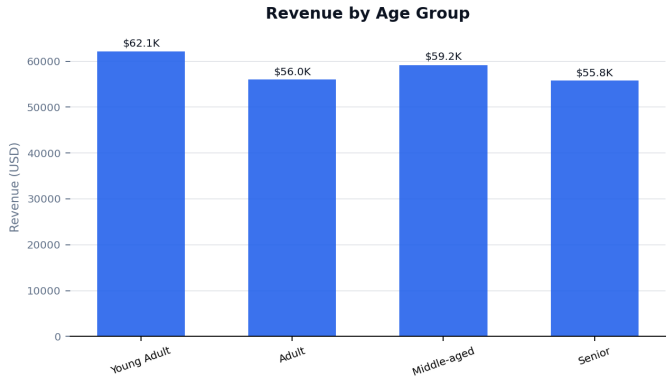


Figure 3: Category revenue distribution.

Figure 4: Subscription status split.



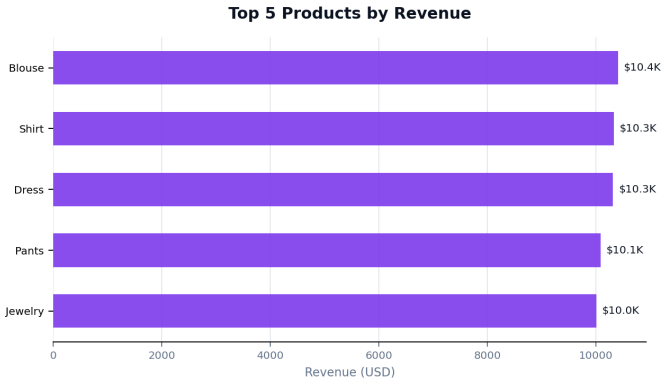


Figure 5: Revenue by age group.

Figure 6: Top products by revenue.

Business interpretation: Clothing is the strongest revenue category in the dataset, non-subscribers represent the larger customer base, and top products such as Blouse, Shirt, Dress, Pants, and Jewelry contribute strongly to revenue.

Dashboard Overview

Power BI Desktop was connected directly to the PostgreSQL database using local import mode. This ensured that the dashboard consumed the cleaned database table instead of the raw CSV file.

Dashboard visuals included:

- KPI Cards: Total Customers, Average Purchase Amount, Average Review Rating.
- Donut Chart: Subscription Status distribution.
- Column and Bar Charts: Revenue and sales by product category and age group.
- Slicers: Interactive filters for category, age group, subscription status, gender, and season.

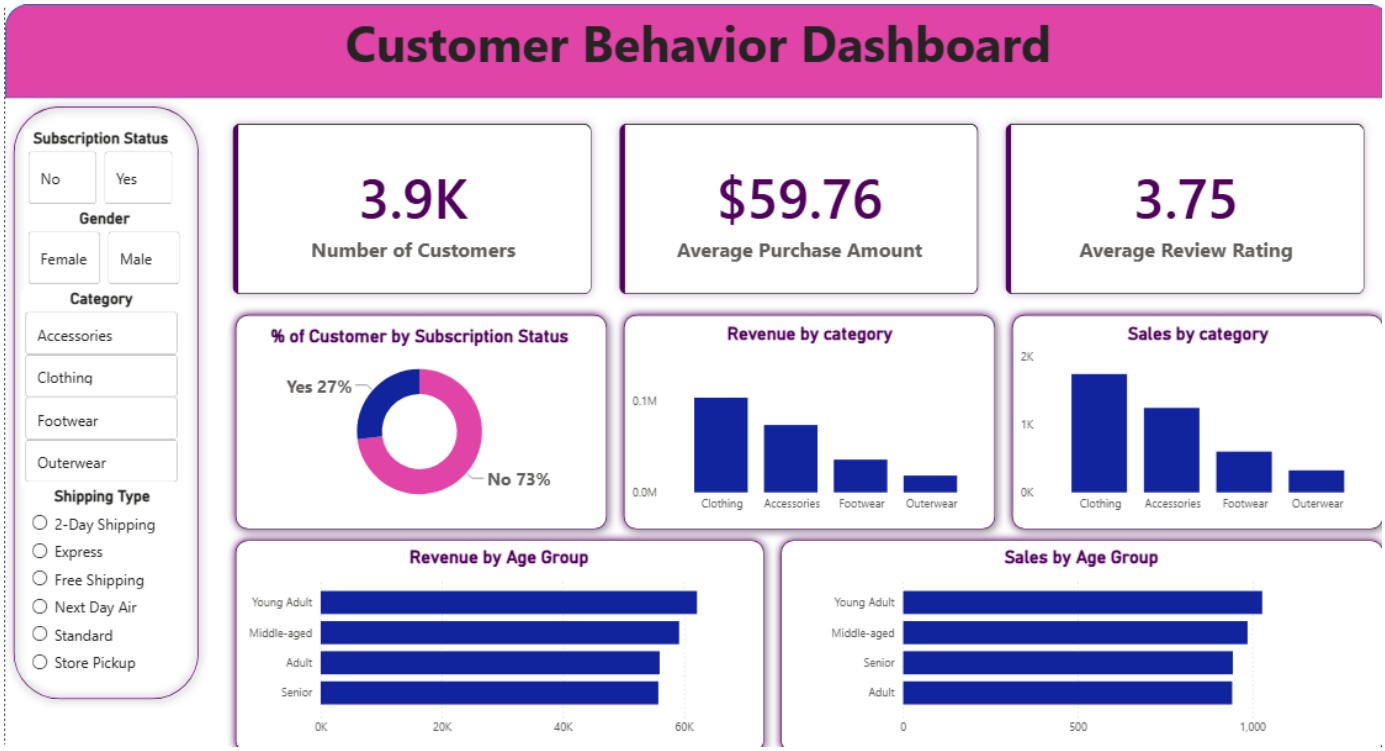


Figure 7: Final Power BI dashboard screenshot with KPI cards, subscription split, category revenue, sales charts, age group analysis, and slicers.

The dashboard was designed for recruiter and engineering manager review, with clear KPIs at the top and analytical breakdowns below. The slicers make the report interactive and allow users to explore customer behavior across multiple dimensions.

Conclusion

Skills Demonstrated

This project successfully delivers an end-to-end analytics and engineering workflow for customer shopping behavior analysis. It starts with raw CSV ingestion, transforms data using Python, stores the clean dataset in PostgreSQL, explores the data using advanced SQL, and presents insights through Power BI.

Skill Area	Evidence from Project
Software Architecture	Designed a layered pipeline from raw data to analytical database to BI dashboard.
Data Engineering	Cleaned data, engineered features, handled missing values, and loaded data into PostgreSQL.
Database Management	Used PostgreSQL as the central storage layer and SQLAlchemy for Python connectivity.
SQL Analytics	Applied aggregations, CTEs, and window functions in DBeaver.
Business Intelligence	Built a Power BI dashboard with KPI cards, charts, and slicers.
Business Storytelling	Analyzed revenue, customer demographics, subscription status, and purchase behavior.

Portfolio positioning:

- Shows practical ability to move from raw data to a decision-ready dashboard.
- Demonstrates both coding depth and business reporting skills.
- Highlights relevant capabilities for Data Analyst, Data Engineer, Analytics Engineer, SQL Developer, and Power BI Developer roles.
- Proves understanding of how Python, SQL, databases, and BI tools work together in a real data workflow.

Overall, this project shows the ability to not only analyze data, but also design and build the architecture required to support reliable, reusable, and business-focused analytics.

Appendix: Mermaid.js Architecture Code

The following Mermaid.js code can be used to recreate the data pipeline architecture diagram in a README, documentation site, or markdown-based portfolio page.

```
flowchart LR
    A[Raw CSV Dataset] --> B[Python + Pandas]
    B --> C[Data Cleaning and Feature Engineering]
    C --> D[SQLAlchemy Engine using URL.create]
    D --> E[(PostgreSQL Database)]
    E --> F[DBeaver SQL Analysis]
    E --> G[Power BI Desktop]
    F --> I[Business Insights]
    G --> H[Interactive Dashboard]
    H --> I
```

PDF export workflow completed: DBeaver SQL screenshot, Power BI dashboard screenshot, and architecture visual have been inserted into the final recruiter-facing portfolio report.